ARCA

AGENTIC RESEARCH CONTEXT ARCHITECTURE

ARCA V2 · SHARED CLI & AGENT SKILLS

Quick Start & User Guide

One interface, from your coding-agent session or your terminal.

01 Connect & set up

02 Initialize the project

03 Work with evidence

04 Checkpoint & resume

> **The idea:** Talk to your coding agent. It uses ARCA's shared CLI to manage the records. You supply the research facts and approve scientific decisions, costly runs, and commits.

1 · Open the project and choose your interface

Copy or clone the ARCA template into a project folder. For an existing repository, merge the scaffold carefully and preserve existing code, data, and instructions.

Open that folder in VS Code and start a **Claude Code or Codex session**. With Remote SSH, open the project on the server and ensure the agent executes commands there. Python 3.9+ must be available in that environment.

Where to type	Example
Claude Code chat	`/arca Show the current state and next action.`
Codex chat	`$arca Show the current state and next action.`
Project terminal	`./arca status`

`/arca` and `$arca` invoke the **project skill in the agent chat**. The agent runs `python3 arca` from the repository root. The skill files are already included; no separate ARCA plugin or Python package installation is needed for this route.

Check the setup before changing anything

> **SETUP CHECK —** Inspect this repository, Git state, ARCA scaffold, and available skills. Check that this agent can discover the project instructions and the arca skill. Report missing or conflicting files. Do not modify anything or run experiments yet.

Prefer direct terminal commands?

```
./arca --help
./arca status
./arca check
```

Use `python3 arca` instead of `./arca` if executable scripts are unavailable. A session needs shell access and tool permissions appropriate to the requested operation.

2 · Initialize the research once per repository

In the agent chat, start with one of these requests:

```
Claude Code: /arca Help me initialize this research project.
Codex:       $arca Help me initialize this research project.
```

The agent should establish the objective, questions, datasets, metrics, compute constraints, and target outputs using information you provide. For a more explicit request, paste:

> **INITIALIZE ARCA —** Read AGENTS.md and the applicable instructions. Ask for the research objective, questions, available datasets, evaluation metrics, compute constraints, and target outputs. After I confirm the plan, use the shared CLI to set the objective and next action, then complete only the relevant records with facts I have supplied. Do not invent findings or launch GPU jobs. Run the structural check and show me the changes before proposing the first Git commit.

What initialization writes

File	Section initialized by the CLI
`research/CHARTER.md`	Research objective
`context/STATE.md`	Current objective
`context/NEXT.md`	One concrete next action under Now

The command fills empty or template sections. If a section contains different project notes, it reports the conflict before writing any target file. The agent can then review the section with you. Other scientific details remain your input.

Initialize directly from the terminal

```
./arca init
```

For an agent or script without interactive input:

```
./arca init \
  --objective "Compare calibration methods on held-out data." \
  --next-action "Inspect data/registry.yaml and define splits."
```

> **Ready when:** the objective and next action are documented, the remaining protocol details are understood, the structural check passes or its issues are explained, and you have reviewed the changes.

Optional: make the bare `arca` command available

```
python3 -m pip install -e .
arca --root /path/to/project status
```

This installation exposes the terminal command in the active Python environment. It does not distribute the project skills or copy a template into another repository. On a remote server, install there if you want that command there.

3 · Use the same interface throughout the day

In the examples below, start with `/arca` in Claude Code or `$arca` in Codex, then describe the operation.

When	What to ask the agent	Result
Resume	“Resume this project and tell me the next action.”	Canonical state and relevant evidence; use arca-resume for the focused workflow.
Prepare	“Prepare an experiment for [hypothesis]. Define baseline, controls, metrics, seeds, and a pilot.”	An E-#### record and completed protocol using known facts; no training launched by the CLI.
Monitor	“Check the actual job, logs, GPU use, and anomalies.”	The agent must inspect the actual process or scheduler; CLI status alone is insufficient.
Audit	Invoke research-evidence-audit with a finding or claim.	Trace to saved metrics, config, data, and artifacts; gaps reported.
Check	“Check the project structure and summarize issues.”	Required-file and selected-reference checks.
Checkpoint	“Checkpoint this session, including the latest verification results.”	The agent updates STATE/NEXT, then saves a snapshot.

> **Typical day:** Resume → Plan → Approve → Run & monitor → Review evidence → Checkpoint.

Terminal equivalents

```
./arca status --full
./arca experiment new "temperature scaling baseline"
./arca check --strict
./arca checkpoint
```

Append `--json` for structured output. Use `--root PATH` to select a project explicitly. Installed `arca` finds a project by searching upward from the current directory for `framework.yaml`.

Know what each operation verifies

- **status:** shows recorded notes, Git state, and experiment labels; job status is unknown until inspected externally.
- **check:** checks structure and selected references; scientific validity, remote artifacts, and CortexDB connectivity need separate checks.
- **checkpoint:** saves the notes already on disk. The complete agent workflow updates the notes first.
- **experiment new:** creates starter files and records a planned experiment. Complete the protocol before any run.

Existing Python helper scripts remain available and delegate to the same implementation. The CLI does not start training or create Git commits.

4 · Common questions

The skill does not appear in my session. What should I check?

Open the correct project folder. Codex needs `.agents/skills/arca/SKILL.md`; type `$` or use `/skills` to select it. Claude Code needs `.claude/skills/arca/SKILL.md`; type `/` to browse. Current Claude versions offer `/reload-skills` for a newly created skills directory. Reopen the agent session if changes have not appeared.

Can I ask the agent without using the skill shortcut?

> From this repository root, run `python3 arca status --json` and explain the current state and next action.

My terminal says “arca: command not found.”

Use `./arca` from the project root or `python3 arca`. The bare command becomes available after installing the package in the active Python environment.

How do I resume after compaction or a model switch?

Invoke `/arca-resume` in Claude Code or `$arca-resume` in Codex, or ask the arca skill to resume. It reads canonical repository records. Resume quality depends on the previous checkpoint.

Does closing VS Code stop training?

A job survives only if its actual server-side process or scheduler setup supports that. ARCA records job information; it does not keep processes alive.

Can Claude and Codex both work on this project?

Use separate tasks or worktrees and avoid editing the same files simultaneously. Allocate experiment IDs serially within a shared working tree. Another agent's review is distinct from an experimental replication.

Optional integrations

CortexDB and Feynman: install both with `init --with cortexdb feynman`, or add them to an existing project using `integrations install`. The next page explains scope, setup, and invocation.

Further reading and invocation references

`docs/CLI.md` — command reference, installation, JSON output, and troubleshooting.
`docs/GUIDELINES_v2.md` — research protocol.
`docs/SKILLS.md` — focused workflows.

Invocation syntax checked on 2026-10-30 against the official Codex skills guide, Claude Code skills guide, and Claude Code VS Code guide. Extension behavior varies by installed version; CLI tests do not establish UI compatibility for every version.

The agents do the work. You make the research decisions. The repository keeps the evidence.

5 · CortexDB memory and Feynman research

Install once, reopen the session, then use the new skills in either agent. Plain `init` keeps working offline without these optional components.

Choose where they should be available

Scope	Skills and memory
`project` · default	Skills in this project; one project database shared by Claude and Codex. Recommended for separate research projects.
`user`	Skills in every project for this user; one shared memory database. Install on each machine where the agents run.

During initial setup, in chat

```
Claude: /arca init --with cortexdb feynman --scope project
Codex:  $arca init --with cortexdb feynman --scope project
```

The agent asks for missing research facts, initializes the records, and installs both integrations for both hosts. Use `--agents claude` or `--agents codex` to configure only one.

Already initialized? Add them from the terminal

```
./arca integrations install cortexdb feynman --scope project
./arca integrations status
```

Use the same request after `/arca` or `$arca` in chat. Add `--dry-run` to preview installation without downloads or writes. Research notes stay unchanged.

> **Next:** Reopen the agent session. Handle its normal MCP trust prompt, then select the new skills. Installation does not grant permissions or start research jobs.

Claude Code chat	Codex chat
`/feynman lit calibration methods`	`$feynman lit calibration methods`
`/feynman review paper/draft.md`	`$feynman review paper/draft.md`
`/cortexdb recall model decisions`	`$cortexdb recall model decisions`

Feynman: original research workflows with an adapter for the current host's tools. This is the skills edition, not the standalone Pi app. Outputs and provenance go under `research/feynman/`; paper drafts go under `paper/drafts/`.

CortexDB: a local MCP server; both hosts use the same database within the selected scope. Default retrieval is lexical; semantic retrieval requires an embedding provider. Automatic recall/capture hooks are not installed.

Pinned downloads are checked with SHA-256. Repeated installation is idempotent; conflicts preserve existing configurations. Status checks local files, not a live connection. Paths, recovery, and upstream sources: `docs/INTEGRATIONS.md`.